

Apps Simplified Use a mesh from day 0

Zero-trust mTLS, reliability, and golden-metric observability for every service, from minute one, not after the first audit.

Agenda

01	Why a service mesh	Cost of the status quo, and why now
02	How it works	Data plane, control plane, the landscape
03	Security & mTLS	Identity, encryption, authorization
04	Resiliency	Timeouts, retries, circuit breaking
05	Observability & control	Golden metrics, tracing, traffic shaping
06	Getting started	Three commands to a live mesh

01

Why a service mesh

Most teams rebuild the same networking, security, and telemetry in every service. A mesh makes it the platform's job.

DEFINITION

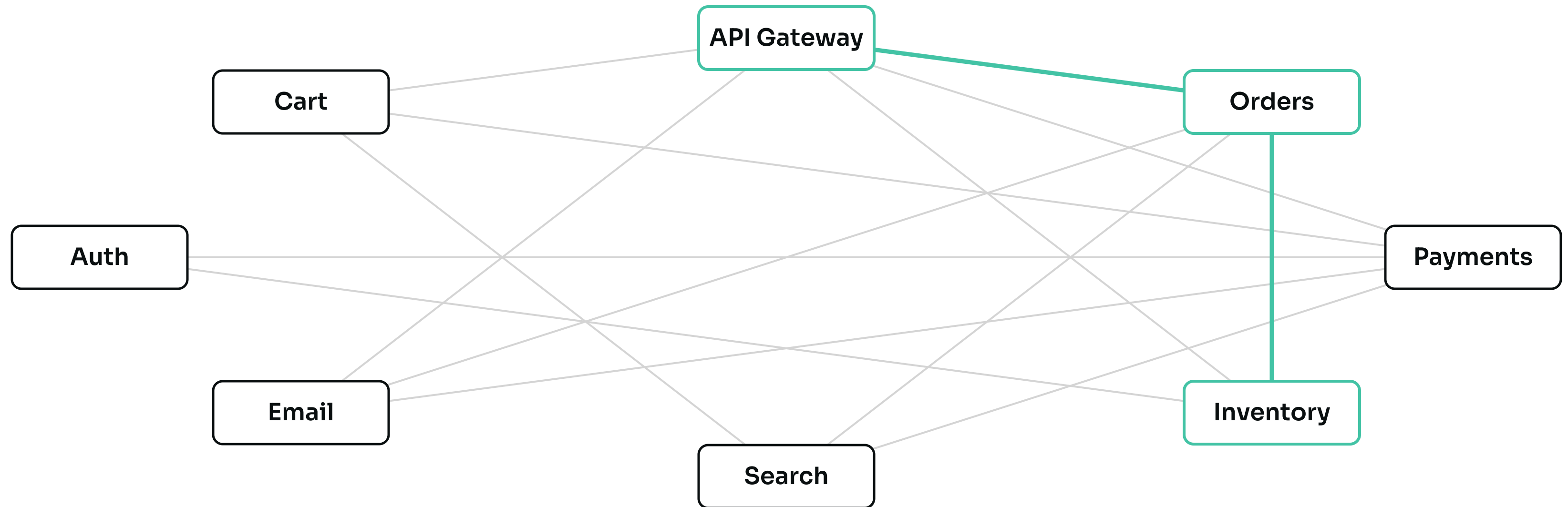
What is a service mesh?

A dedicated **infrastructure layer** for service-to-service communication, an array of **lightweight proxies** deployed alongside your application code, **without the application needing to be aware.**

WHY A MESH

THE PROBLEM

A web of calls



Every new service multiplies the connections. mTLS, retries, timeouts, and metrics get re-solved on **every single edge**, by hand.

WHY A MESH

THE PROBLEM

Every team rebuilds the same code

Team A	Team B	Team C	Team D
Metrics	Metrics	Metrics	Metrics
Retries	Retries	Retries	Retries
TLS & certs	TLS & certs	TLS & certs	TLS & certs
Policy	Policy	Policy	Policy
Tracing	Tracing	Tracing	Tracing

Four teams, four implementations of the same cross-cutting concerns, **inconsistent**, and **four times the maintenance**.

WHY A MESH

Operations, decoupled from code

Proxy

Intercepts every inbound and outbound request

Load balancing

Spreads requests across healthy endpoints

Policy enforcement

Who may call what, evaluated per route

Metrics & logs

Uniform golden metrics for every service

mTLS & identity

Encrypts traffic; gives each workload an identity

Retries & timeouts

Recovers from transient, partial failures

Traffic routing

Splits, mirrors, and shifts traffic

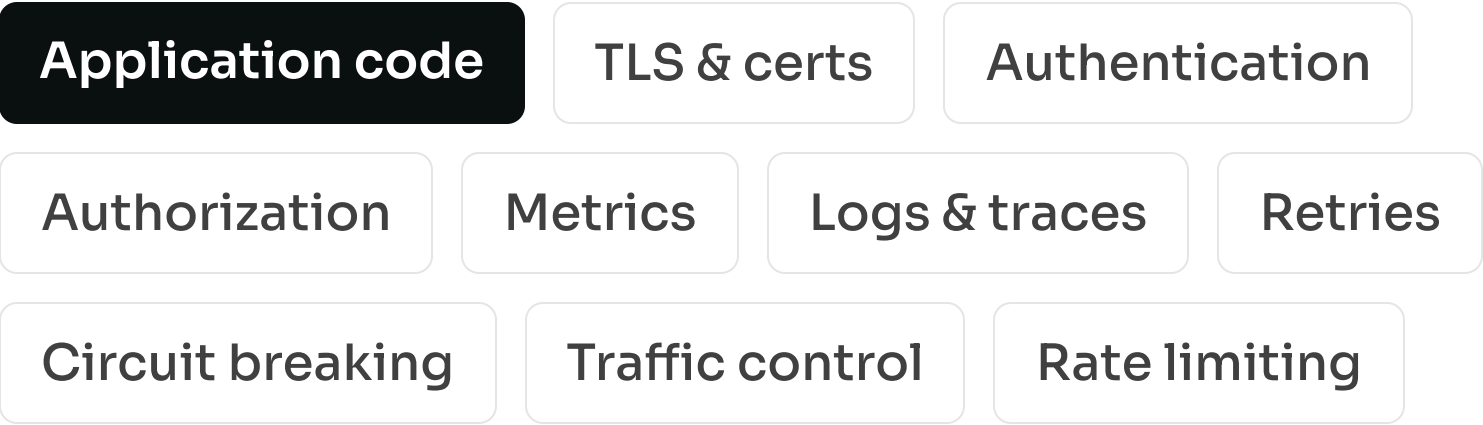
Tracing

Distributed traces, out of the box

BEFORE / AFTER

Reduce complexity

WITHOUT A MESH, THE DEVELOPER OWNS

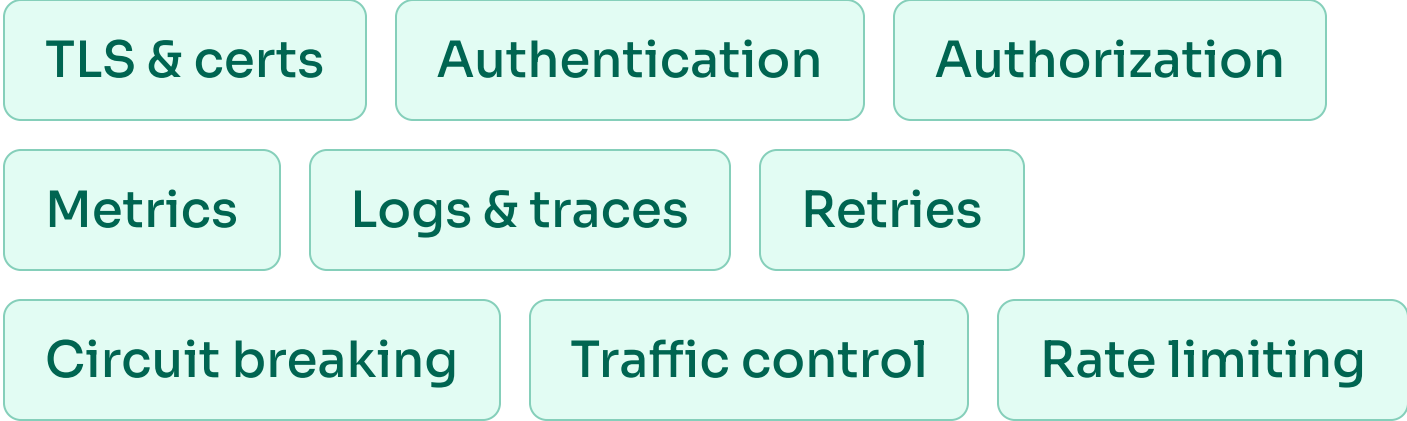


WITH A MESH

You write



The mesh handles



WHY A MESH

Kubernetes isn't a sealed box

Kubernetes schedules containers and gives them a network. It does **not** encrypt that network, verify who is talking, or limit what they may do.

No encryption

Pod-to-pod traffic is plaintext by default

No real identity

A token, not a cryptographic peer identity

No request authz

NetworkPolicies gate connectivity, not calls

THE SERVICE MESH MARKET

41.3%

Projected CAGR, 2025–2033 · USD 0.31B (2024) → **USD 7.14B (2033)**

Data breaches

Attacks push teams to harden application-level security

Compliance

Regulations demand encryption, auth, and audit trails

Complexity

More microservices need consistent, simple controls

THE ARGUMENT

Start on day 0

01 Runway is precious

Every sprint hand-rolling mTLS, retries, or dashboards is a sprint not spent on product-market fit. The mesh gives you all three from minute one.

02 Two engineers ≠ two platform teams

The same people who ship features get enterprise-grade security, reliability, and observability for free, no three new CNCF projects to learn.

03 Avoid the retrofit tax

Waiting until the first security audit usually costs 2–3× more effort than adding the mesh on a blank canvas.

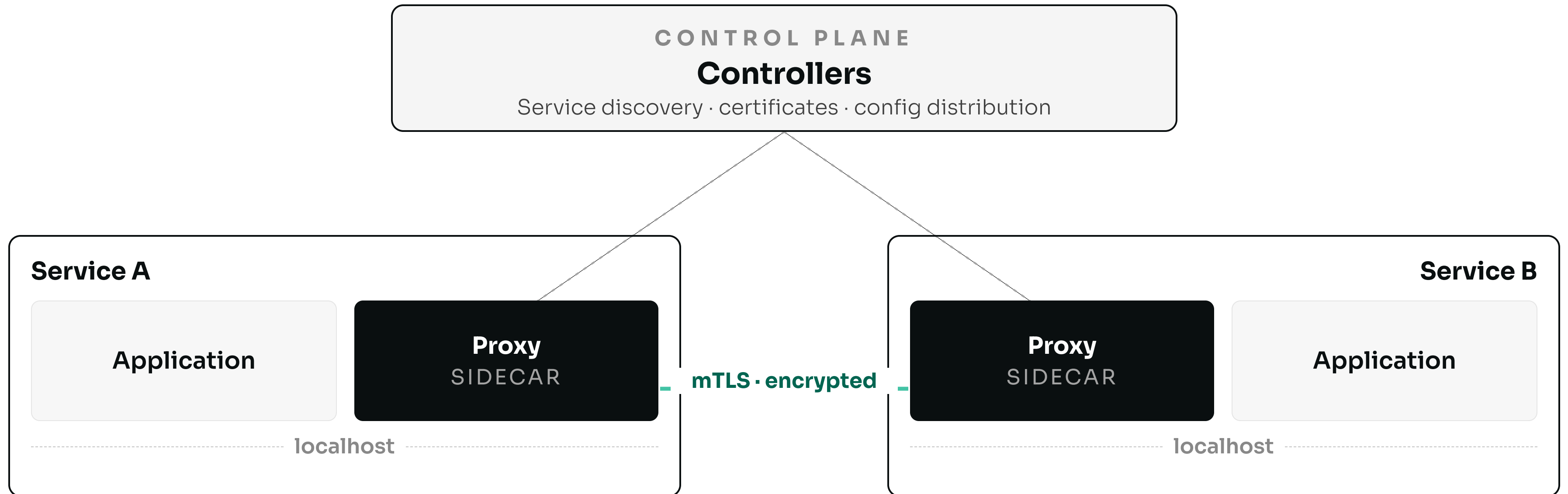
WHY A MESH

02

How it works

Two cooperating layers, proxies in the data path, controllers out of it, and a field of projects that implement them.

Data plane & control plane

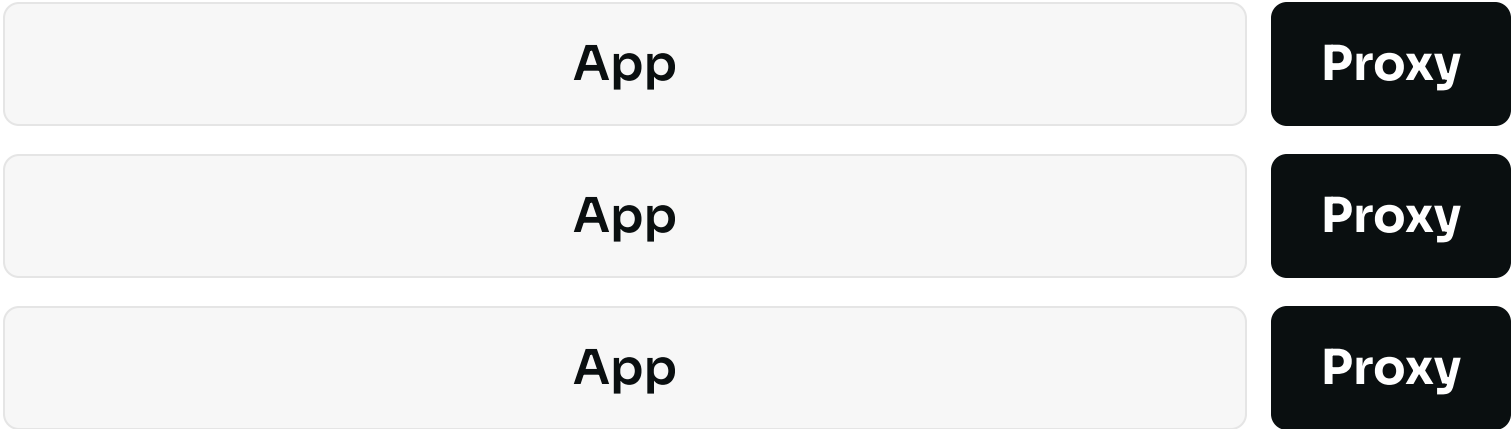


Proxies carry and encrypt every request on **localhost**; the control plane configures them and **never sits in the request path**.

Two data-plane models

Sidecar

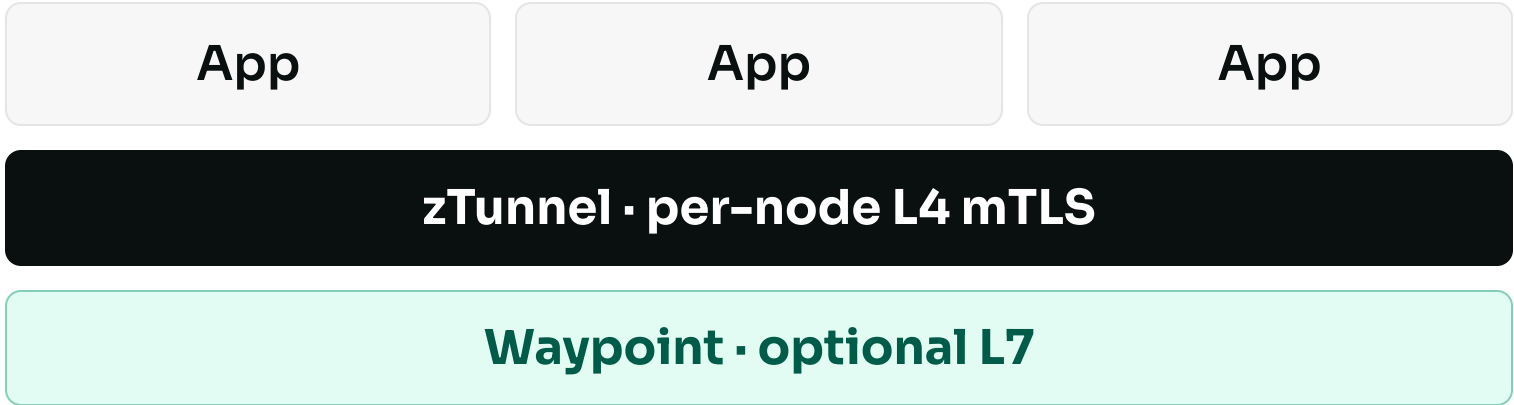
proxy per pod



Mature, simple mental model · per-pod isolation, at the cost of one proxy per pod.

Ambient

shared, sidecar-less · Istio / Cilium



No per-pod sidecar, lighter. Add L7 features via a waypoint only where needed.

Overhead tracks your **traffic**

There's no fixed mesh tax, the proxy's cost scales with what flows through it.

DRIVER · 01

Throughput

CPU rises with requests per second, then falls back as traffic drops.

DRIVER · 02

Payload

Larger request and response bodies mean more bytes to buffer per hop.

DRIVER · 03

Concurrency

Open connections, endpoints, and routes set the memory floor.

The service mesh landscape

Istio

The feature-rich incumbent. Envoy-based, now with an ambient mode.

Linkerd

CNCF-graduated. A Rust micro-proxy built for simplicity and speed.

USED IN THIS TALK

Consul

HashiCorp's mesh, tightly integrated with Nomad and the HashiCorp stack.

Kong

Built on Kuma. Multi-zone, with a gateway heritage.

Cilium

eBPF-based. Mesh features at the kernel layer, sidecar-free.

THE PAYOFF

Four pillars

01

Security

Identity and encryption for every workload, enforced by the proxy.

02

Resiliency

Timeouts, retries, and circuit breaking, configured, not coded.

03

Observability

Uniform golden metrics, traces, and proxy telemetry, for free.

04

Control

Per-route traffic shaping, splits, mirrors, and rate limits.

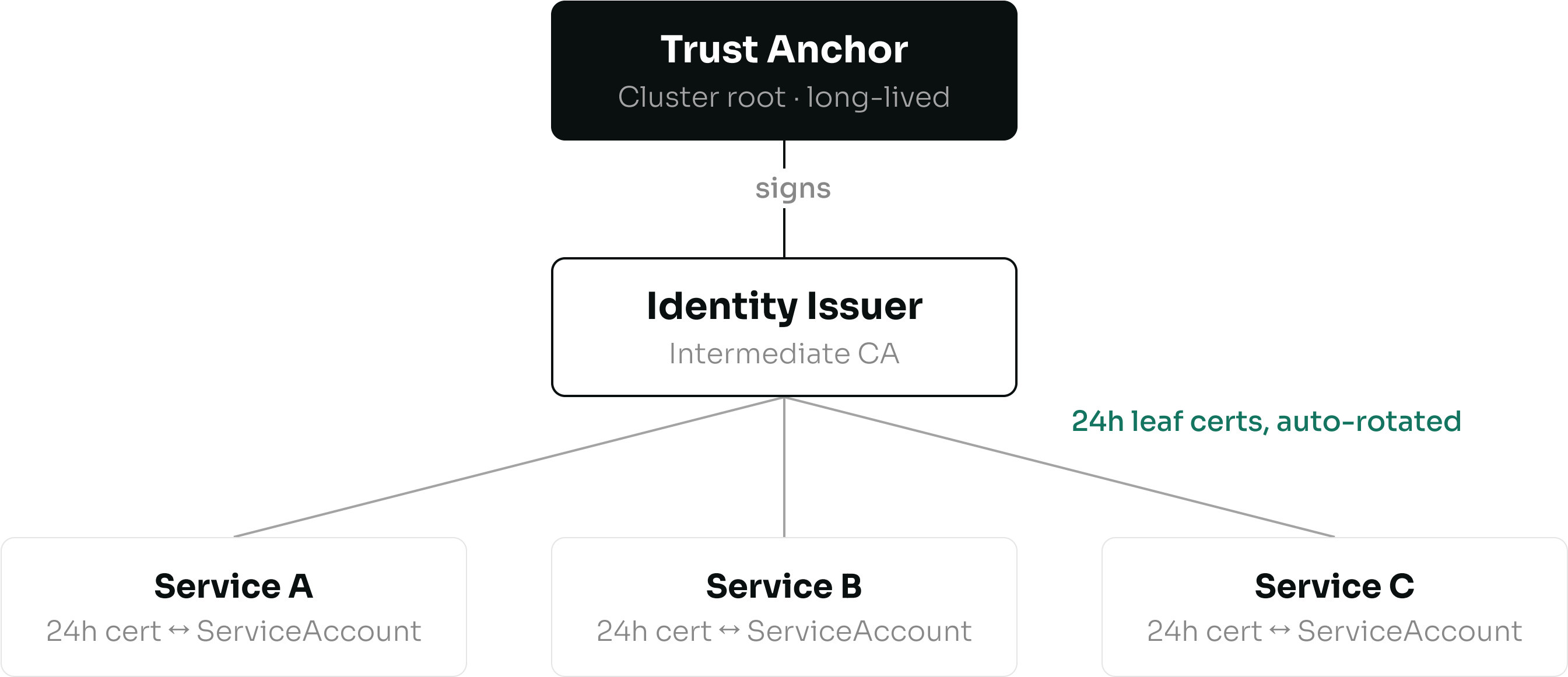
HOW IT WORKS

03

Certificates & mTLS

Every workload gets a verifiable identity and encryption in transit, enforced by the proxy, with zero application changes.

Certificate hierarchy



Issuing identity, enforcing mTLS

1

Pod deployed

An injector webhook adds an init container and a proxy to any annotated pod.

2

Traffic rerouted

The init container rewrites routing so all traffic flows through the proxy.

3

Keypair generated

The proxy creates a keypair bound to the pod's ServiceAccount.

4

CSR sent

A signing request goes to the identity controller with the ServiceAccount token.

5

Cert issued

The controller verifies the account, signs a 24-hour cert, auto-rotated, and returns it.

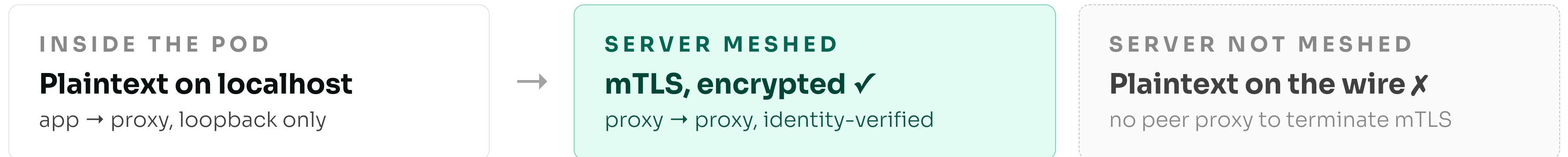
6

mTLS established

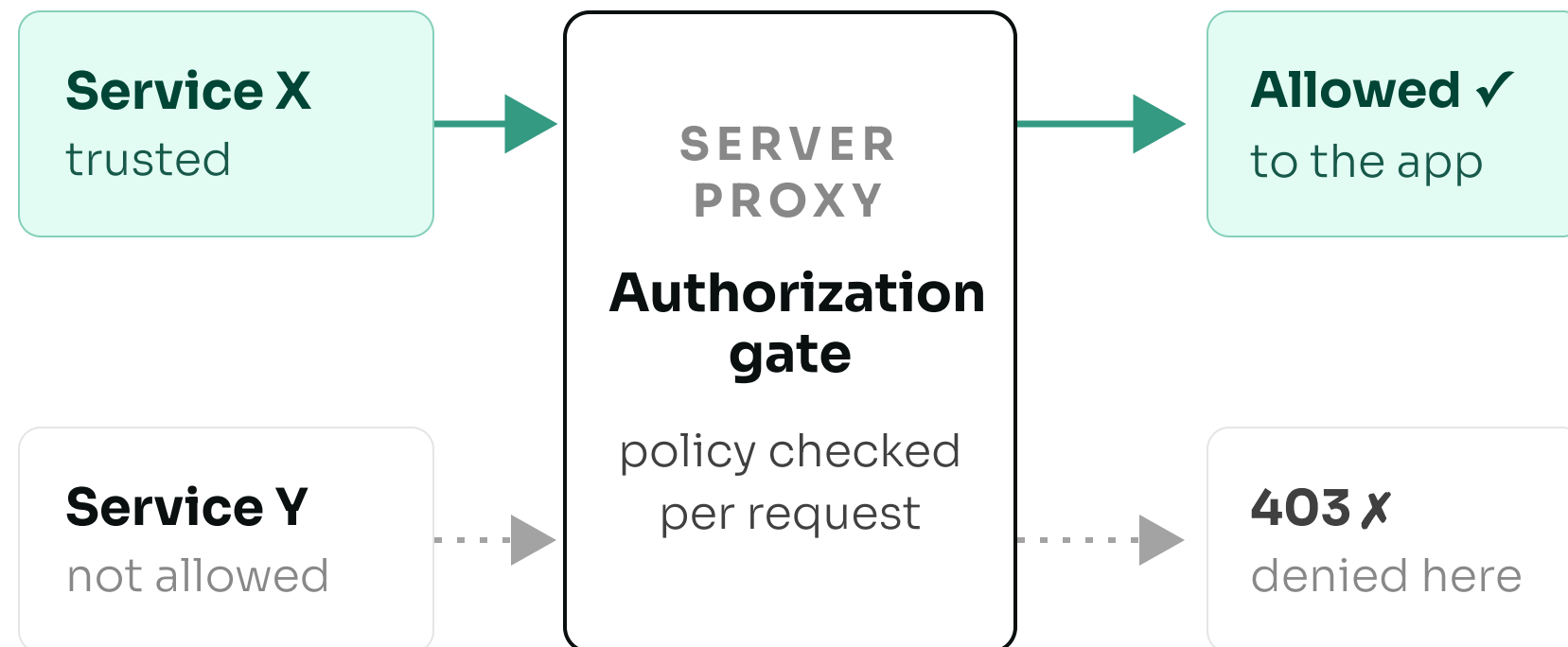
Both proxies present certs, handshake, and encrypt every request.

Encrypted by default

The proxy intercepts the plaintext connection your app already makes and wraps it in **mutual TLS**, no library, no code change.



Allowed, or denied at the door



```
kind: AuthorizationPolicy
spec:
  targetRef: { kind: Server, name: api }
  requiredAuthenticationRefs:
    - { kind: MeshTLSAuthentication }
    # or NetworkAuthentication
```

targetRef, the Server this policy guards

requiredAuthenticationRefs, MeshTLS or Network identities a client must present

Default-deny is a cluster setting (proxy.defaultInboundPolicy); this policy is the allowlist on top.

Authenticating clients

NetworkAuthentication

Authenticate clients that fall within one of the specified IP ranges.

MATCHES
CIDRs

MeshTLSAuthentication

Authenticate clients that present one of the specified mesh identities.

MATCHES
ServiceAccounts

AuthorizationPolicy

Bind one or more authentication requirements to a target.

TARGETS
Server · Route · Namespace

04

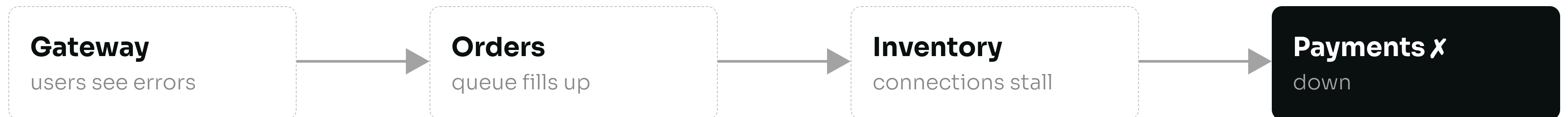
Resiliency

Failures are a given. The mesh contains them, with timeouts, retries, and circuit breaking, configured rather than coded.

WHY IT MATTERS

Cascading failures

Failures are inevitable. The only question is whether **one failure stays local**, or takes the whole call graph with it.



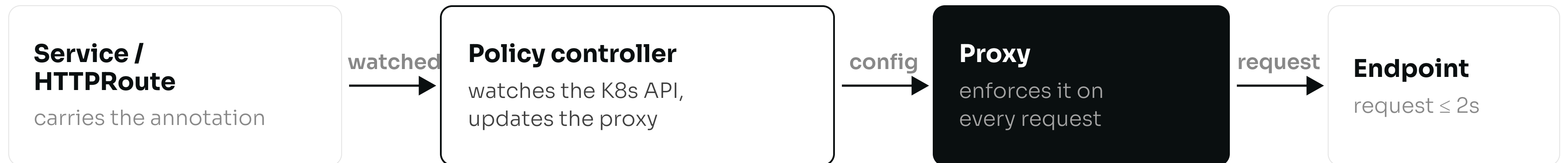
One slow dependency exhausts connections all the way up the graph. **The mesh's job is to keep the fault local.**

RESILIENCY

Timeouts

ANNOTATE THE SERVICE OR HTTPROUTE

```
timeout.linkerd.io/request: 2s  
timeout.linkerd.io/response: 1s  
timeout.linkerd.io/idle: 5s
```

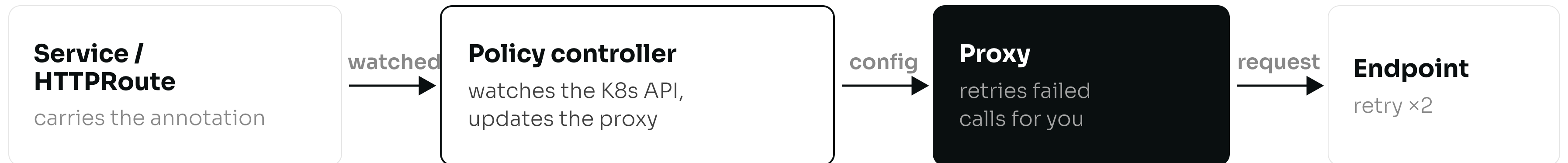


request bounds the whole stream, **response** a backend response, **idle** an idle stream. Enforced by the outbound proxy, so the client must be meshed.

Retries

ANNOTATE THE SERVICE OR HTTPROUTE

```
retry.linkerd.io/http: 5xx  
retry.linkerd.io/limit: "2"  
retry.linkerd.io/timeout: 300ms
```

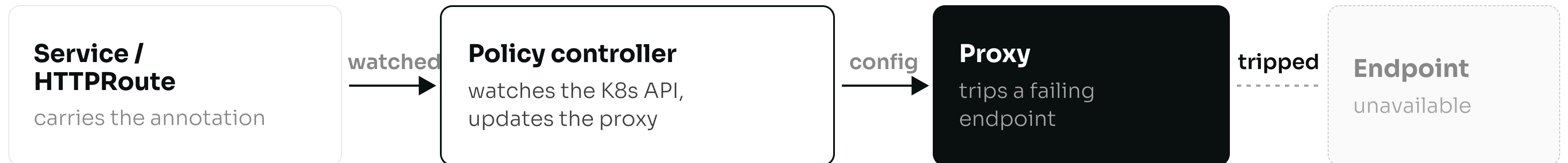


http lists the retryable codes, **limit** caps the retries (default 1), **timeout** the per-try deadline. Retries are bounded so a partial outage can't amplify into a retry storm, and Linkerd also supports retry budgets.

Circuit breaker

ANNOTATE THE SERVICE OR HTTPROUTE

```
balancer.linkerd.io/failure-accrual: consecutive  
balancer.linkerd.io/failure-accrual-consecutive-max-failures: "2"  
balancer.linkerd.io/failure-accrual-consecutive-min-penalty: 300ms
```



Each proxy tracks failures per endpoint; after N in a row it **trips** and stops sending traffic, then probes again after a penalty that backs off exponentially.

05

Observability & control

Uniform golden metrics and traces for free, plus per-route control over exactly how traffic flows.

Golden metrics, for free

SUCCESS RATE

99.4%

Share of successful requests in the window, 1 minute by default, per route and per backend.

TRAFFIC

1.2k_{rps}

Requests per second, how much demand is placed on a given service or route.

LATENCY

p50/95/99

Time to serve requests, split into 50th, 95th, and 99th percentiles per route.

Operational metrics

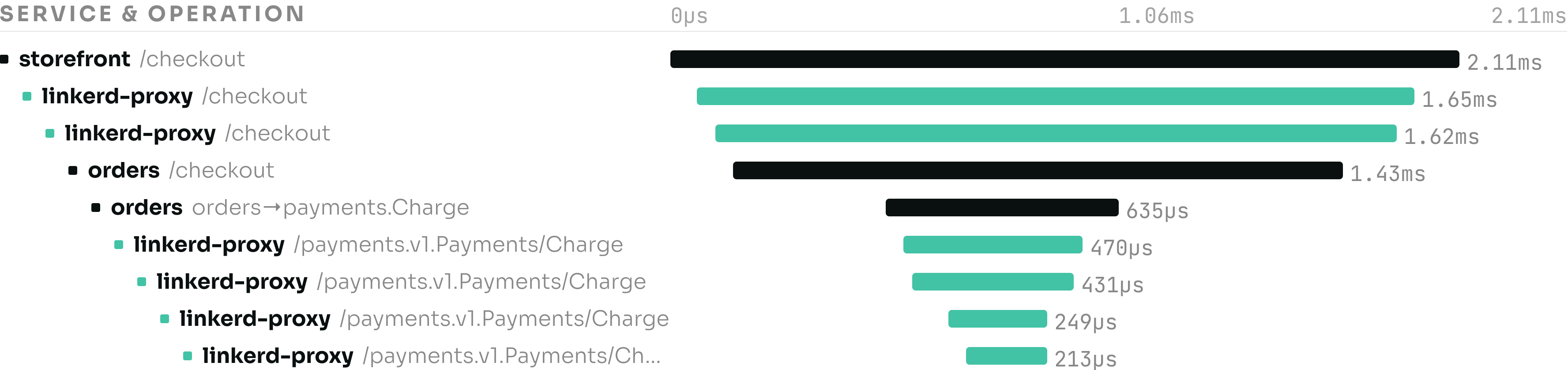
Every proxy exposes `/metrics`, Prometheus scrapes it, Grafana renders it. Nothing to wire up.

HTTP, HTTP/2, gRPC	TCP	Control plane
request_total response_total response_latency_ms ...	tcp_open_total tcp_open_connections tcp_read_bytes_total tcp_write_bytes_total tcp_close_total	control_policy_balancer_endpoints control_dns_resolutions ...

HTTP series carry an `inbound_ / outbound_` prefix in PromQL.

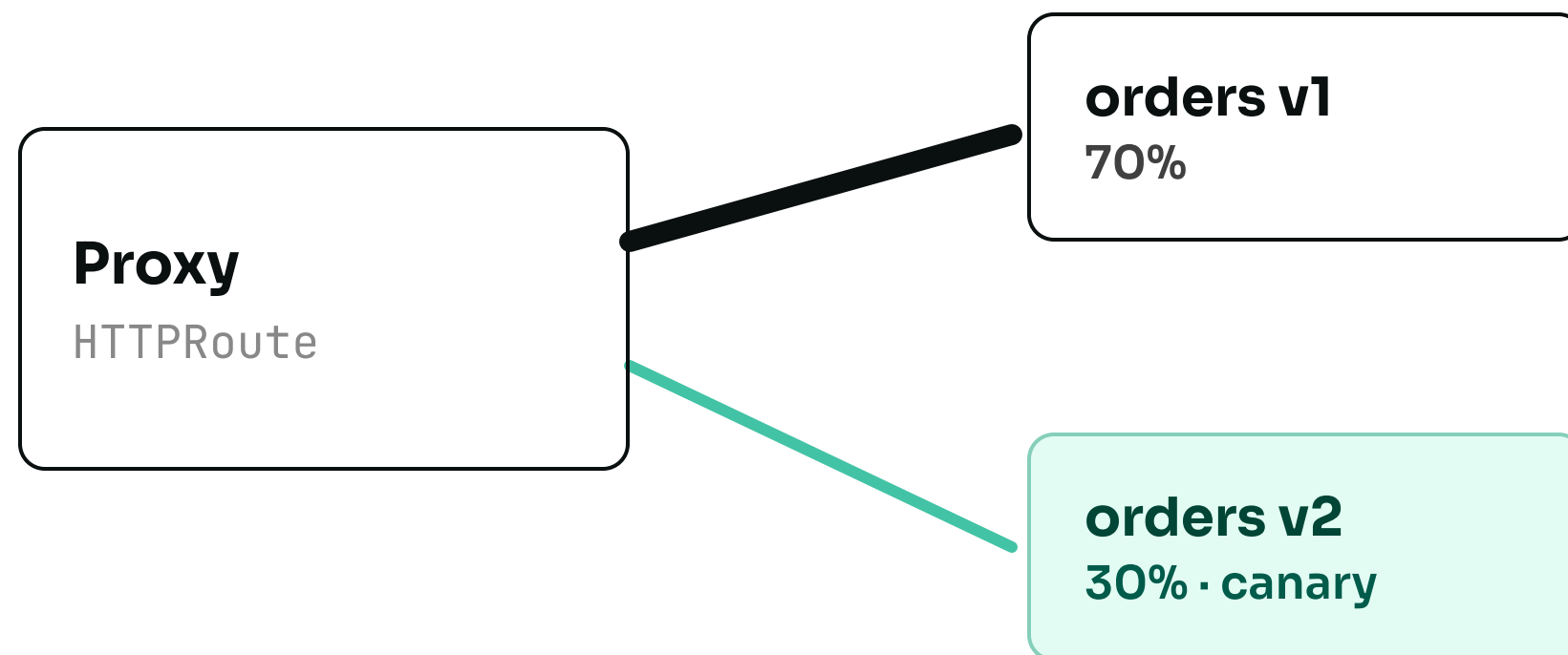
Distributed tracing, out of the box

SERVICE & OPERATION



The instant the **linkerd-proxy** sees a tracing header on a request, it emits its own span. Propagate that one header through your app and the whole call tree appears in Jaeger, with no tracing SDK in your services.

Traffic management



THE HTTPROUTE

```
kind: HTTPRoute
spec:
  rules:
    - backendRefs:
      - { name: orders-v1, weight: 70 }
      - { name: orders-v2, weight: 30 }
```

BEHAVIOR DEPENDS ON `parentRef`

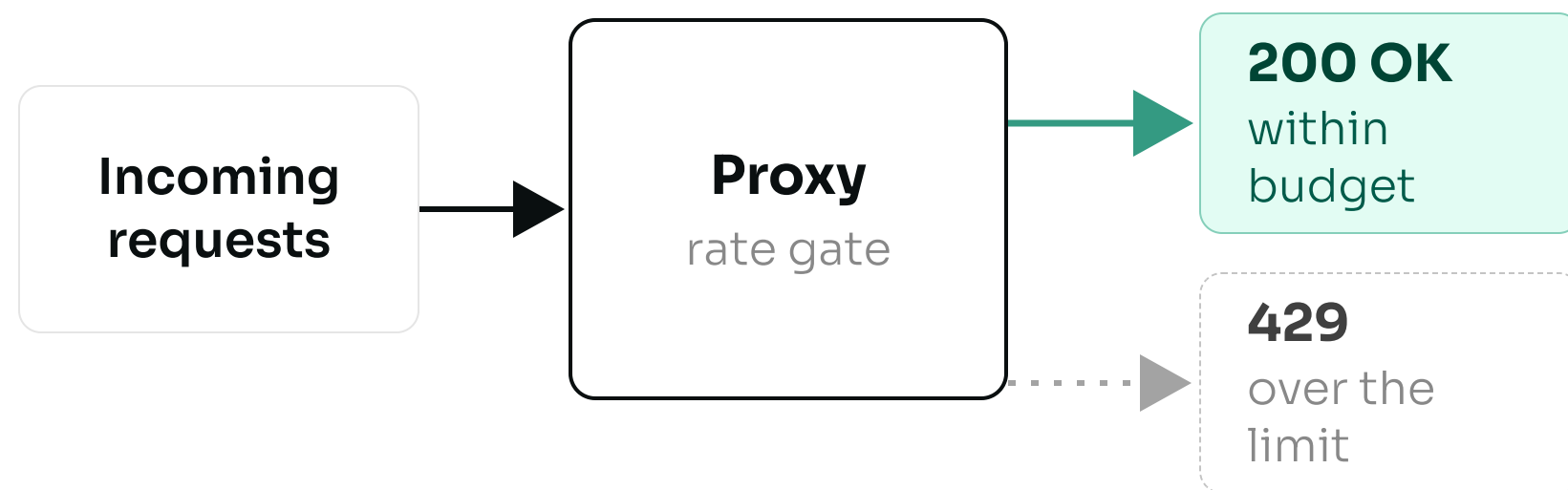
Service, split, redirect, rewrite headers

Server, bind authorization policies

EgressNetwork, route & monitor outbound

Gateway, ignored by the proxy

Local rate limit



```
kind: HTTPLocalRateLimitPolicy
spec:
  targetRef: { kind: Server, name: api }
  total: { requestsPerSecond: 1000 }
  identity: { requestsPerSecond: 100 }
  overrides:
    - requestsPerSecond: 200
      clientRefs: [{ kind: ServiceAccount, name: web }]
```

total, overall cap on the target

identity, per-identity cap, advisory $\leq$ total

overrides, a list of per-client limits, each with clientRefs

06

Getting started

From an empty cluster to a meshed workload in three commands.

Mesh in three commands

linkerd-install, zsh

```
# install with the CLI
```

```
$ curl --proto '=https' --tlsv1.2 -sSfL https://enterprise.buoyant.io/install | sh
```

```
$ linkerd install --crds | kubectl apply -f -
```

```
$ linkerd install | kubectl apply -f -
```

```
$ linkerd check
```

```
control plane is healthy ✓
```

```
# or with Helm charts
```

```
$ helm install linkerd-enterprise-crds linkerd-buoyant/linkerd-enterprise-crds
```

```
$ helm install linkerd-enterprise-control-plane linkerd-buoyant/linkerd-enterprise-control-plane
```

LIVE DEMO

See it running

Scan to deploy the demo on your own cluster
and browse the full source.

shorturl.at/yvSTI



SCAN TO TRY IT

Thank you

Terima kasih · 감사합니다 · ありがとう · Gracias